📘 Universal Serial Bus (USB)

Universal Serial Bus (USB) is a common technology used to connect devices like keyboards, mouse, printers, flash drives, and phones to a computer or even to each other. It was developed in 1995 by big companies like Intel, Microsoft, and IBM to make connecting devices simple and easy. Before USB, connecting devices was complicated, but USB made it possible to use one standard port for many devices.

The main idea of USB is to provide a simple way to add external devices (called peripherals) to a computer. It uses a **host–device system**, where the computer acts as the host and controls communication, while other devices act as peripherals. USB also allows multiple devices to be connected using hubs, making it expandable. It uses strong cables and connectors, and the system automatically detects and configures devices when they are plugged in.

Advantages of USB

- Many devices can be connected using a **single standard port**
- No need for **manual settings** like DIP switches or IRQs
- **Low cost** and easy to expand
- Supports **auto-configuration (plug-and-play)**
- Supports **hot-plugging** (connect without restarting)
- Provides **good performance and speed**
- Can supply **power to devices** (no extra adapter needed)

Limitations of USB

- **Speed limitation** compared to some technologies like Gigabit Ethernet
- No **direct device-to-device communication** (host is required)
- Limited **cable length (about 5 meters)**
- Does not support **broadcasting** (only one-to-one communication)

Working Principle of USB

- A **host (computer)** controls all communication
- Devices (keyboard, mouse, etc.) act as **peripherals**
- When a device is plugged in, the system **automatically detects and configures it**
- Data is transferred using a **master-slave protocol** (host = master, device = slave)
- USB hubs allow **multiple devices** to connect to one port

USB Functionality

1. Power Delivery (Paragraph)

USB can provide power to connected devices. This means many devices like keyboards, flash drives, and even phones can work or charge without needing a separate power supply.

2. Battery Charging 1.2 (Paragraph)

USB also supports charging devices like smartphones. A normal USB port gives small power, but newer standards like BC 1.2 allow more power for faster charging. Devices can also communicate with the charger to receive the correct amount of power safely.

3. USB On-The-Go (OTG)

USB OTG allows mobile devices like smartphones and tablets to act like a computer (host). This means you can connect devices like a mouse, keyboard, or flash drive directly to your phone using an OTG adapter.

4. DisplayPort Alt Mode (Paragraph)

This feature allows USB-C cables to carry both data and video signals. So, you can connect your laptop or phone to monitors or TVs using the same USB cable without needing extra drivers.

5. USB Docking (Paragraph)

USB docking stations increase the number of ports available on a laptop. They allow you to connect multiple devices like monitors, keyboard, mouse, and internet cable through one USB connection. It is very useful for work setups.

USB Cable Types, Ports & Color Coding

USB comes in different types of connectors like Type-A, Type-B, Micro USB, and USB-C. These connectors are used for different devices. Sometimes USB ports are color-coded (like blue, black, or orange), but these colors are not standard and may vary between manufacturers. They usually indicate speed or special features like charging.

USB Connector Pinouts

Explanation (Easy Paragraph):USB is a **serial communication system**, which means data is sent one bit at a time through wires. A standard USB cable has **4 wires** inside. Two wires are used for **power supply** (+5V and Ground), and the other two wires are used for **data transmission**, called **D+ and D-**. These two data wires work together as a **pair (twisted pair)** to reduce noise and improve signal quality, especially over longer distances.

USB uses **half-duplex communication**, meaning data can move in both directions, but **not at the same time**. The D+ and D- lines are not separate; they always work together using **differential signaling**, which helps reduce errors caused by external noise.

Pin Configuration (USB Cable)

- **Pin 1 → +5V Power (Red)**
- **Pin 2 → Data - (White / Yellow)**
- **Pin 3 → Data + (Green / Blue)**
- **Pin 4 → Ground (Black / Brown)**

USB Data Transfer Speeds

Explanation (Easy Paragraph):USB supports different data speeds depending on the version. Slower speeds are used for simple devices like keyboards, while higher speeds are used for devices like storage drives and cameras.

Types of USB Speeds

- **Low Speed → 1.5 Mbps** (keyboard, mouse)
- **Full Speed → 12 Mbps** (common devices)
- **High Speed → 480 Mbps** (USB 2.0 devices)
- **SuperSpeed → 5 Gbps** (USB 3.0 and above)

Extra Concept (Easy Paragraph):
When you connect a USB device, it tells the system its speed by setting either the **D+ or D- line to a higher voltage (3.3V)**. This helps the computer detect the device and understand how fast it can communicate. If no signal is detected, the system assumes no device is connected.

USB Connector Types (Pinouts)

USB Type-A and Type-B

Explanation (Paragraph): Type-A is the most common USB connector (used in computers), while Type-B is usually used for printers and larger devices. Both have **4 pins** for power and data.

Mini USB & Micro USB

Explanation (Paragraph):
Mini and Micro USB are smaller connectors used in older mobile devices, cameras, and accessories. They have **5 pins**, including an extra **ID pin** used for features like OTG (On-The-Go).

USB Type-C

Explanation (Paragraph):USB Type-C is the modern and most advanced connector. It is **reversible**, so you can plug it in any direction. It supports **fast data transfer, power delivery, and video output**, making it a universal connector for modern devices.

USB Standard 3 (SuperSpeed)

Explanation (Easy Paragraph):USB 3.0, also called **SuperSpeed USB**, is a major improvement over older versions. It provides much faster data transfer and better performance. It uses **extra pins** (total 9 pins instead of 4) to allow separate paths for sending and receiving data, which increases speed and efficiency. Features of USB 3.0

- Data speed up to **5 Gbps or more**
- Uses **separate paths** for sending and receiving data
- Better **power management**
- Devices can **notify before sending data**
- Has **extra 5 pins** for high-speed communication

USB Type-C & USB4

USB Type-C is the connector shape, while USB4 is the latest technology standard. USB4 improves speed, power, and performance. It supports very high data rates and can handle both **data and video at the same time**. It also works with older USB versions, so it is backward compatible. Features of USB4

- Minimum **20 Gbps**, up to **40 Gbps** speed
- Supports **multiple protocols** (USB, PCIe, DisplayPort)
- **Dynamic bandwidth allocation** (data + video together)
- Supports **fast charging up to 100W**
- Compatible with older USB versions

USB vs USB-C vs Thunderbolt

Many people get confused between USB4 and USB-C. USB-C is just the **shape of the connector**, while USB4 is the **technology inside**. Thunderbolt is another high-speed technology similar to USB4, but usually offers slightly better performance and features like connecting multiple devices and monitors.

- **USB-C → Connector type (shape)**
- **USB4 → Technology (speed & features)**
- **Thunderbolt → Advanced high-speed technology**

What's New in USB4?

Explanation (Easy Paragraph):USB4 is the latest version of USB technology and it brings many improvements in speed, flexibility, and performance. It allows very fast data transfer (up to 40

Gbps), supports multiple types of data (like video and storage), and improves how devices share bandwidth. USB4 also works with older USB versions, so you can still use your previous devices.

What is Protocol Tunneling?

Explanation (Easy Paragraph): Normally, devices communicate using a specific "language" called a **protocol**. For communication to work, both devices must understand the same protocol. Protocol tunneling is a smart method where one protocol is used like a **pipe (channel)** to carry another protocol's data inside it.

In USB4, this means that **USB-C can carry other types of data like DisplayPort (video) or PCIe (high-speed data)** through the same cable. This makes USB4 very powerful because one cable can handle many types of communication at the same time, without needing separate connections.

What is USB4 Fabric?

Explanation (Easy Paragraph): The term **fabric** is used to describe a network where many parts are connected together, like threads in cloth. In USB4, fabric means a system where multiple devices and data types are connected and managed together efficiently.

USB4 fabric allows different types of data (like video, storage, etc.) to **share the same connection dynamically**. This means the system can automatically adjust and give more bandwidth to the task that needs it most, improving performance.

Will Apple Support USB4?

Explanation (Easy Paragraph): Yes, Apple supports USB4 in its newer devices like MacBooks and Mac Mini. Even though Apple uses its own processors (Apple Silicon), it still supports both **USB4 and Thunderbolt 3**, so users can enjoy high-speed connections and compatibility.

USB-C and USB 2.0

Explanation (Easy Paragraph):USB-C and USB 2.0 are not the same thing. **USB-C is just the shape of the connector**, while USB 2.0 is a **technology standard** that defines speed and features.

A USB-C cable can support USB 2.0, USB 3.x, or even USB4, depending on the cable quality. For example, a USB-C cable with USB 2.0 will have slower speed (480 Mbps) but may still work for charging. So, when buying a cable, you should check both **data speed and power capacity**.

USB Architecture

Explanation (Easy Paragraph):USB uses a structure called **tiered-star topology**. At the center is the **host (computer)**, which controls everything. The host connects to **USB hubs**, and hubs connect to other hubs or devices like mouse, keyboard, or printer.

The host is responsible for managing all communication. Devices cannot talk directly to each other; they must go through the host. This ensures proper control and avoids data conflicts.

Working Principle of USB Architecture

- There is **only one host (master)** in the system
- All other devices act as **peripherals (slaves)**
- Communication is always **started by the host**
- Devices **cannot communicate directly** with each other
- Data transfer happens only when **host requests it**

Types of Connections in USB Hubs

- **Upstream Connection** → connects hub to higher-level hub or host
- **Downstream Connection** → connects hub to devices or lower-level hubs

Important Features of USB Architecture

- Supports up to **127 devices** using 7-bit addressing
- Maximum **7 levels (tiers)** of connection
- Each hub has **1 upstream port** and multiple downstream ports
- Maximum cable length is **5 meters per segment**
- Total distance can be extended using hubs (up to ~30 meters)
  USB is designed for **short-distance communication**, mainly for devices near the computer. If long-distance communication is needed, technologies like Ethernet are more suitable.

Host Controllers (USB)

Explanation (Easy Paragraph): In a USB system, the **host controller** is a part of the computer hardware that controls all USB operations. It works together with the **root hub** to manage connected devices. However, programmers do not directly control the hardware; instead, they use a software layer called the **Host Controller Device (HCD)**. This acts as a bridge between the operating system and the USB hardware.

The important thing to remember is that the USB standard defines **how data is transferred**, but it does not define **how the internal hardware communicates with the computer**. That part is handled by different host controller standards.

Types of Host Controller

- **OHCI (Open Host Controller Interface)**
  - Hardware-based design
  - Developed by Compaq, Microsoft, etc.
  - Supports low speed and full speed

- **UHCI (Universal Host Controller Interface)**
  - Software-based design
  - Developed by Intel
  - Requires Intel license
  - Supports low and full speed
- **EHCI (Extended Host Controller Interface)**
  - Used for USB 2.0
  - Handles high-speed data (480 Mbps)
  - Uses OHCI/UHCI for slower devices
- **VHCI (Virtual Host Controller Interface)**
  - Virtual controller (no physical hardware)
  - Used to share USB devices over networks
- **USB4 Host Interface**
  - Advanced controller for USB4
  - Manages multiple protocols (USB, DisplayPort, PCIe, Thunderbolt)

Endpoints

Explanation (Easy Paragraph): Endpoints are the **points where data enters or leaves a device**. They are like small memory areas inside a USB device that store data for sending or receiving. Communication in USB always happens between the **host and endpoints**.

Each device has multiple endpoints, but when a device is first connected, only **Endpoint 0** is active. This is used for initial setup (called enumeration). After configuration, other endpoints are activated as needed.

Pipes

Explanation (Easy Paragraph):A pipe is a **logical connection** between the host software and a device endpoint. It is like a communication path through which data flows. Pipes are created when a device is connected and removed when it is disconnected.

Types of Pipes

- **Message Pipes**
  - Two-way communication (bi-directional)
  - Used for control transfers

    - Controlled by host
- **Stream Pipes**
    - One-way communication (uni-directional)
    - Used for bulk, interrupt, and isochronous transfers
    - Can be controlled by host or device

Transfer Types

Explanation (Easy Paragraph):USB supports different types of data transfer depending on the need. Some transfers focus on accuracy, while others focus on speed or timing.

Types of Transfer

- **Control Transfer**
    - Used for setup and configuration
    - Exchanges commands and status
- **Bulk Transfer**
    - Transfers large data
    - Not time-sensitive
    - Has error correction
- **Interrupt Transfer**
    - Small data, needs quick response
    - Used by keyboard, mouse
- **Isochronous Transfer**
    - Time-critical data (audio/video)
    - No error correction
    - Ensures continuous data flow

Transactions

Explanation (Easy Paragraph):A USB transaction is a **complete communication step** between host and device. It usually consists of up to **three packets**. The host always starts the transaction, and data is exchanged in an organized way to ensure correctness.

Working Principle of Transaction

- **Token Packet** → sent by host (tells what to do)
- **Data Packet** → carries actual data
- **Handshake Packet** → confirms success or failure

USB Packets & Format

Explanation (Easy Paragraph):USB communication happens in the form of **packets**, which are small units of data sent between host and device. All data is sent **bit by bit (serially)**, starting from the least significant bit (LSB). Each packet contains several fields that help identify and verify the data.

Important Packet Fields

- **Sync Field** → Synchronizes sender and receiver
- **PID (Packet ID)** → Identifies packet type
- **ADDR** → Device address (supports up to 127 devices)
- **ENDP** → Endpoint number
- **CRC** → Error checking
- **EOP** → End of packet signal
  In simple terms, USB works like a **controlled communication system** where the host manages everything. Data moves through endpoints using pipes, in the form of packets, and different transfer types are used depending on the situation. This organized system ensures reliable and efficient communication between the computer and connected devices.

Handshaking (USB)

Explanation (Easy Paragraph): Handshaking is a method used in USB communication to **check whether data is successfully sent or not**. It helps avoid data loss and ensures that communication between the host and device is correct. After sending data, the receiver sends a response signal to confirm whether the data was received properly or not.

Terms Used in Handshaking

- **ACK (Acknowledgement)** → Data received successfully
- **NAK (Negative Acknowledgement)** → Device not ready / no data
- **STALL** → Request not supported or device stopped
- **NYET (Not Yet)** → Device is busy, not ready for next data
- **ERR** → Error occurred in transaction

- **No Response** → No reply from device

USB Device States

Explanation (Easy Paragraph): When a USB device is connected to a computer, it goes through several states before becoming fully usable. These states help the system properly identify, configure, and manage the device.

Device States

- **Attached State** → Device is physically connected
- **Powered State** → Device receives power from host (max 100 mA)
- **Default State** → Device reset, uses address 0
- **Addressed State** → Device gets a unique address
- **Configured State** → Device is ready to work (driver loaded)
- **Suspended State** → Device goes to low power mode (idle > 3 ms)

Device Classes

Explanation (Easy Paragraph):USB device classes are used to **identify the type of device** and load the correct driver automatically. This allows devices from different companies to work properly if they follow the same class standard.

Types of Device Classes

- **Human Interface Device (HID)** → Keyboard, mouse, joystick
- **Mass Storage** → Flash drive, hard disk
- **Audio** → Microphone, speaker
- **USB Hub** → Expands USB ports
- **Image** → Scanner, webcam
- **Printer** → Printers, CNC machines
- **Wireless** → Bluetooth, Wi-Fi adapters

USB Electrical Signaling

Explanation (Easy Paragraph):USB sends data using **electrical signals** through two wires called **D+ and D-**. These signals are sent in opposite directions (differential signaling), which helps reduce noise and improve signal quality. Data is transmitted in a special format called **NRZ (Non-Return-to-Zero)**.

To maintain proper communication, USB uses a technique called **bit stuffing**, where an extra 0 is added after six consecutive 1s. This helps keep the signal synchronized.

Working Principle of Electrical Signaling

- Data sent using **D+ and D- lines (differential signals)**
- Uses **NRZ format** for transmission
- **Bit stuffing** prevents synchronization loss
- Uses **voltage levels** to represent 0 and 1
- Host and device use **pull-up and pull-down resistors** to detect connection
- Signal states like **J, K, and SE0** indicate different conditions

Extra Easy Idea (Paragraph): When no device is connected, both data lines stay low (SE0 state). When a device is connected, it pulls one line high to show its speed. Communication then happens by switching between signal states. For high-speed devices, a special process called **chirping** is used to switch to faster mode.USB communication is a well-organized system where the host controls everything. Handshaking ensures safe data transfer, device states manage connection steps, device classes help identify devices, and electrical signaling ensures accurate data transmission. All these parts work together to make USB reliable and efficient.

Enumeration (USB)

Explanation (Easy Paragraph): Enumeration is the process by which a USB device becomes **recognized and usable by the computer (host)**. When you plug in a USB device, the system needs to identify what the device is, what it can do, and how to communicate with it. This process is called enumeration.For enumeration to work properly, the device must have **firmware** (internal code) that can send its information to the host. On the computer side, a **device driver** must be available so the operating system understands how to use that device. Once everything is set up correctly, the device becomes ready to use through applications.

How USB Devices Communicate (Enumeration Process)

Explanation (Easy Paragraph): When a USB device is connected, the host starts communication by sending a **reset signal**. This helps detect the device and its speed. Then the host reads information from the device and assigns it a **unique address**. After that, the required driver is loaded, and the device is configured. From this point, the device is ready to send and receive data.This process happens not only when a device is plugged in, but also when the computer starts or when a device is removed or reconnected.

Working Principle of Enumeration

- Host sends a **reset signal** to the device
- Device speed is detected
- Host reads device information

- Device is assigned a **unique address (7-bit)**
- Required **driver is loaded**
- Device enters **configured state** (ready to use)

Communication Using Endpoints & Pipes

Explanation (Easy Paragraph): In USB, communication happens between the **host and endpoints** inside the device. Endpoints act as sources or destinations of data. Each endpoint has a unique address and direction (input or output).

The connection between host and endpoint is called a **pipe**. Even though physically devices are connected through hubs, logically it feels like each device has a direct connection with the host.

The host controls all communication and continuously checks (polls) devices to see if they need to send or receive data.

Bandwidth Management

Explanation (Easy Paragraph):USB manages data transfer using a concept called **bandwidth**, which is divided into small time units called **frames**. Each frame lasts **1 millisecond**. The host controls how this bandwidth is shared among devices.Time-critical devices like audio and video (isochronous and interrupt) get priority and can use up to **90% of total bandwidth**. The remaining bandwidth is used by control and bulk transfers.

Important Points of Bandwidth

- USB divides data into **frames (1 ms each)**
- Each frame contains **1500 bytes**
- **Isochronous & Interrupt → high priority (up to 90%)**
- **Control & Bulk → use remaining bandwidth**
- Host manages all bandwidth allocation

What is a Descriptor?

Explanation (Easy Paragraph):A descriptor is a **data structure (information block)** that a USB device sends to the host during enumeration. It tells the host everything about the device, such as its type, capabilities, number of endpoints, and how communication should happen.Descriptors are very important because the host uses this information to **configure the device and create proper communication channels (pipes)**. Types of Descriptors

- **Device Descriptor**
    - Basic information about the device
    - Includes vendor ID, product ID

- **Configuration Descriptor**
    - Describes power requirements and configurations
- **Interface Descriptor**
    - Defines device functionality (like audio, HID, etc.)
- **Endpoint Descriptor**
    - Provides details about each endpoint (direction, type)
- **String Descriptor**
    - Contains human-readable info (manufacturer name, product name)

Enumeration is the **starting step of USB communication**, where the device introduces itself to the host. Using descriptors, the host understands the device and sets up communication through endpoints and pipes. The host then controls all data transfer using bandwidth and frames, ensuring smooth and efficient communication.